
ORACLEPROTO: A REPRODUCIBLE FRAMEWORK FOR BENCHMARKING LLM NATIVE FORECASTING VIA KNOWLEDGE CUTOFF AND TEMPORAL MASKING

Yiding Ma* Chengyun Ruan* Kaibo Huang† Zhongliang Yang† Linna Zhou
Beijing University of Posts and Telecommunications
{yidingma, ruanchengyun815, huangkaibo, yangzl, zhoulinna}@bupt.edu.cn

ABSTRACT

Large language models are moving from static text generators toward real-world decision-support systems, where forecasting is a composite capability that links information gathering, evidence integration, situational judgment, and action-oriented decision making. This capability is in broad demand across finance, policy, industry, and scientific research, yet its evaluation remains difficult: live benchmarks evaluate forecasts before answers exist, making them the cleanest way to measure forecasting ability, but they expire once events resolve; retrospective benchmarks are reproducible, but they cannot reliably distinguish genuine forecasting from facts a model may have already learned during pretraining. Prompting models to "pretend not to know" cannot replace a genuine knowledge boundary. We propose OracleProto, a reproducible framework for evaluating LLM native forecasting capability. OracleProto reconstructs resolved events into time-bounded forecasting samples by combining model-cutoff-aligned sample admission, tool-level temporal masking, content-level leakage detection, discrete answer normalization, and hierarchical scoring. Instantiated on a FutureX-Past-derived dataset with six contemporary LLMs, OracleProto distinguishes forecasting quality, sampling stability, and cost efficiency under controlled information boundaries, while reducing residual leakage to the 1% level, an order of magnitude below tool-only temporal filtering. OracleProto turns LLM forecasting from one-off evaluation into an auditable, reusable, and trainable dataset-level capability, providing a unified interface for fair cross-model comparison and a controlled signal source for downstream SFT and RL. Code and data are available at <https://github.com/MaYiding/OracleProto> and <https://huggingface.co/datasets/MaYiding/OracleProto>.

1 Introduction

Large language models (LLMs) are increasingly entering real-world decision-support settings. Forecasting requires models to connect information search, evidence integration, situational judgment, and action-oriented decision making into a complete reasoning chain under conditions where information is incomplete, evidence continues to evolve, and outcomes have not yet been revealed, rather than merely retrieving known facts from memory. Native forecasting capability is one of the most representative high-level composite capabilities of large language models. At the same time, there is broad and urgent real-world demand for this capability in domains such as financial risk assessment, policy analysis, public safety, and industrial planning. Therefore, forecasting capability has become a key task for evaluating whether large language models possess a usable world model and the ability to reason under uncertainty. Prior work shows that language models (LMs) can achieve forecasting performance comparable to that of competitive human forecasters [3], while dynamic forecasting benchmarks have begun to treat LLM forecasting over future events as a distinct evaluation objective [4, 13].

The difficulty of evaluating forecasting capability arises from time itself. Prospective evaluation on unresolved events offers the strongest contamination control, since the true answer does not yet exist when a forecast is submitted. Such

*Equal contribution.

†Corresponding authors.

evaluation, however, requires waiting for events to resolve, question sets become obsolete over time, and repeated testing under identical questions and information conditions is difficult. Retrospective evaluation on resolved events is easier to audit and compare, but it is also especially vulnerable to mistaking factual recall for forecasting capability. This tension creates the central paradox of LLM forecasting evaluation: the most credible questions are difficult to replay, while the most replayable questions can easily lose their forecasting character. FutureX-Past illustrates this problem clearly. The dataset preserves resolved forecasting questions that can support behavioral analysis and reinforcement learning, but it also cautions that historical outcomes may have entered the training data of newer models and therefore cannot be directly used to evaluate live future prediction capability [2].

Whether resolved events can become forecasting tasks again depends on whether the evaluation system can re-establish clear time boundaries around the information available to the model.

Existing research has made the premise that large language models can participate in forecasting sufficiently credible, but it also reveals two remaining gaps. First, much of the observed forecasting performance still depends on external harnesses, such as retrieval pipelines, aggregation procedures, and calibration layers, rather than on the model’s native forecasting capability itself. Second, the field still lacks a dataset-level evaluation framework that is auditable, reusable, and comparable across models and time. Retrieval-augmented forecasting systems organize search, evidence integration, forecast generation, and aggregation into end-to-end pipelines, enabling language models to approach the performance of human collectives on real prediction-platform questions [3]. ForecastBench and FutureX establish contamination-free live evaluation standards by dynamically generating and continuously updating future-event questions [4, 13]. Time-R1, outcome-based reinforcement learning, and OpenForecaster/OpenForesight show that forecasting capability can be continually optimized through training [1, 6, 10]. These studies collectively show that forecasting is a valuable and optimizable capability, yet they still lack a dataset-level evaluation framework that is auditable, reusable, and comparable across models and time.

We propose OracleProto, a reproducible framework that benchmarks the native forecasting capability of large language models by reshaping the evaluation object at the dataset level through knowledge cutoffs and temporal masking. Events that have already occurred, but lie beyond the knowledge boundary permitted to the model, are reorganized into discrete forecasting tasks that are reproducible and scorable. The model’s knowledge cutoff defines the boundary of potentially available parametric knowledge, while temporal masking constrains the boundary of external information retrieval. Under these constraints, revealed events are no longer historical questions carrying contamination risk, but replayable forecasting samples. This framework separates models, datasets, and evaluation protocols. Forecasting performance is no longer merely a one-time result from a live competition or a particular agent evaluation framework, but becomes a dataset-level evaluation problem that can be recorded, revisited, audited, and accumulated.

OracleProto further serves capability evaluation, data construction, and model training. It enables researchers to systematically compare the native forecasting capability of large language models on questions with known answers, while reducing the risk of mistaking factual recall for forecasting. More importantly, under this framework, any expired forecasting benchmark, such as FutureX-Past [2, 13], archived ForecastBench questions [4], or historical Metaculus records [7], can be reactivated as contamination-free forecasting training samples according to the target model’s knowledge cutoff date. Expired data are no longer discarded artifacts, but instead constitute a monotonically growing pool of training corpora over time. As a result, forecasting is no longer a one-off outcome, but becomes an accumulable data asset.

This work makes the following contributions:

- **A formal dataset-level framework for LLM forecasting evaluation.** This work reconstructs the evaluation of LLM-native forecasting capability from one-time outcomes dependent on live events into dataset-level tasks that are definable, auditable, and reproducible. The framework provides not only a dataset, but also a continuously extensible method for sample construction.
- **The OracleProto evaluation protocol.** The protocol combines knowledge cutoffs, temporal masking, leakage detection, answer normalization, and hierarchical scoring to turn revealed events into controlled forecasting tasks. It exposes the same information boundaries and the same scoring interface to every model under test, so cross-model differences in measured accuracy track model behaviour rather than shifting evaluation conditions.
- **A systematic evaluation benchmark and a trainable forecasting framework.** Based on FutureX-Past, we construct a leakage-controlled forecasting evaluation set and evaluate six contemporary LLMs. The experiments show that the framework can jointly measure forecasting quality, sampling stability, cost efficiency, and residual leakage risk, while supplying signals for native training of forecasting capability through SFT, RL, and forecasting agents.

2 Related Work

2.1 Forecasting Systems and Dynamic Benchmarks

The premise that LLMs can play a serious role in forecasting is settled, in the first instance, by Halawi et al. [3], who show that retrieval-augmented language models approach the crowd aggregate of competitive forecasters on a static snapshot of questions opened after their LMs’ mid-2023 knowledge cutoff, and by MIRAI [12], which evaluates LLM agents on international-event forecasting through structured historical events, news, and tool interfaces. These works establish feasibility, but their static-snapshot designs become contamination-prone as model knowledge cutoffs advance, which is the failure mode the next generation of benchmarks is built to avoid.

ForecastBench [4] refreshes its question set on a biweekly schedule and admits only events whose answers do not yet exist, achieving prospective contamination protection at the cost of impermanence: each biweekly question set is superseded by the next as resolved items exit the rolling question bank, so the leaderboard is a one-way temporal stream rather than a reusable artifact. FutureX [13] extends anti-contamination measures and difficulty stratification under the same live-evaluation philosophy, and is paired with a frozen subset, FutureX-Past [2], intended for retrospective behavioral analysis, reinforcement learning, and retrieval evaluation; its dataset card warns, however, that historical outcomes may already have entered newer models’ training data and that the subset must not be used as an ordinary live-prediction benchmark. FutureX-Past therefore exposes the central problem we attack: a frozen forecasting corpus does not become a reusable evaluation object simply by being archived. Reuse requires a dataset-level mechanism that re-establishes the model’s true knowledge boundary on every replay, which neither static snapshots nor live benchmarks provide.

2.2 Forecasting as a Trainable Capability

A second line of work treats forecasting as a trainable objective. Time-R1 [6] stages a rule-based reward curriculum over historical news, while outcome-based reinforcement learning on Polymarket questions [10] improves probabilistic calibration through market-resolved future outcomes; together they establish that forecasting capability is end-to-end optimizable against time-stamped, verifiable outcomes. The closest neighbor to our work in this line is OpenForecaster and OpenForesight [1], which synthesize large-scale open-ended forecasting questions from daily news and use an offline news corpus for both training and retrieval in order to suppress future-information leakage; this is, in effect, the closest existing practice to retrospective, leakage-aware training, although its evaluation remains tied to a single vendor and a single cutoff. A growing set of further training-oriented studies extends the same direction, with cross-paper comparability limited by private corpora, private cutoffs, and private retrieval stacks. The bottleneck is therefore not whether forecasting can be trained but what it is trained against: every system supplies its own dataset, retrieval pipeline, and validation protocol, so the field still lacks a reusable, model-agnostic data object that other groups can audit, reuse, and extend across model knowledge cutoffs.

2.3 Information Boundaries in Forecasting Evaluation

A third line of work diagnoses, rather than builds, evaluation protocols, and its central message is that prompt-time discipline cannot substitute for a real knowledge boundary. Paleka et al. [9] show that label noise alone suffices to invalidate strictly proper scoring, so answer verification at the dataset level becomes a hard requirement rather than merely a quality-control choice. Li et al. [5] sharpen this picture from a different angle, exhibiting a substantial and systematic gap between *simulated ignorance*, where a model is prompted to forget, and *true ignorance*, where it never knew in the first place; reasoning-optimized models fare worse on this gap, which forces the knowledge cutoff to be enforced as a sample-admission condition rather than as a prompt instruction. BLF [8] achieves a residual leakage rate below 1.5% on a backtested ForecastBench evaluation through a four-layer date-leakage defense, yet defends only a single inference and a single backtest, so any change in backtest window, API, or question set requires the entire result to be redone. OracleProto carries the same discipline into the dataset schema and the run unit defined in Section 3, so that model-aligned knowledge cutoffs admit samples, multi-layer temporal masking gates retrieval, and verified discrete answers anchor scoring; the same corpus thus remains reproducible across models, teams, and calendar years. BLF and OracleProto are complementary, the former constrains single-inference behavior, while the latter standardizes the reusable forecasting corpus itself.

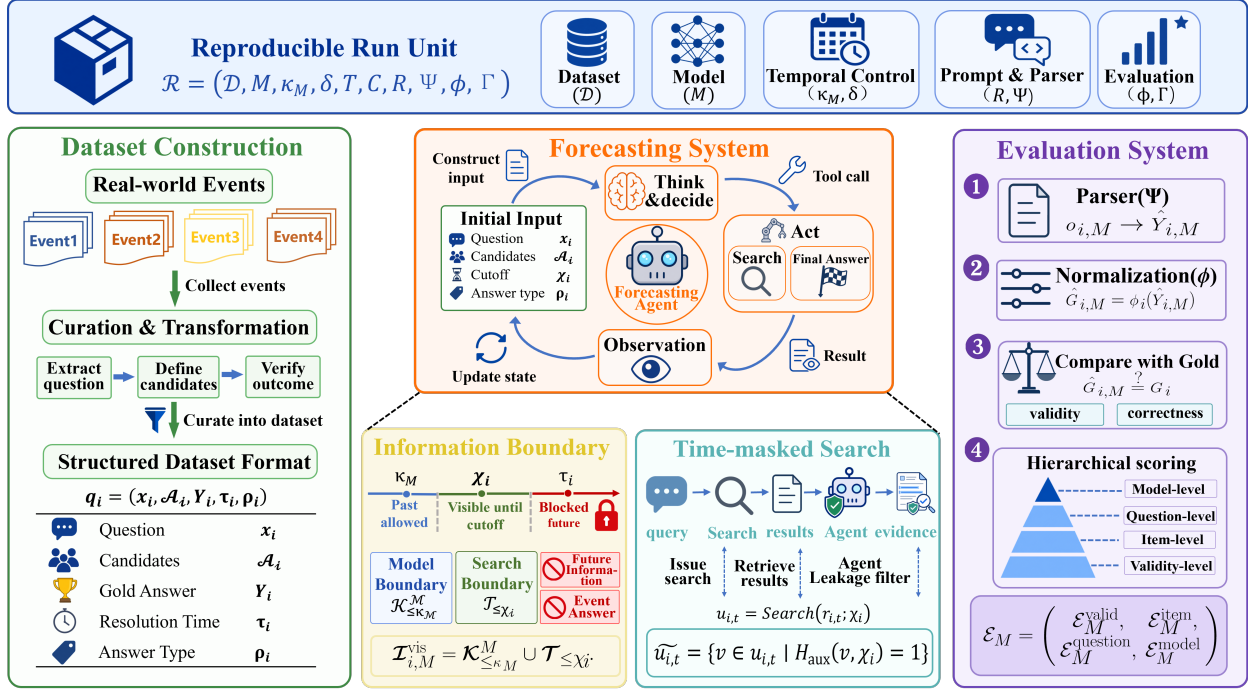


Figure 1: OracleProto overview. Centered on the reproducible run unit $\mathcal{R} = (\mathcal{D}, M, \kappa_M, \delta, T, C, R, \Psi, \phi, \Gamma)$, the framework specifies the dataset, model, knowledge cutoff, temporal offset, interaction rule, and search budgets, prompt renderer, parser, normalization rule, and scoring protocol. Given a resolved real-world event, in accordance with the principles of OracleProto, resolved real-world events are reconstructed into a dataset of structured forecasting instances, each comprising pending forecasting questions, candidate answers, verified gold outcomes, and event occurrence times. At inference time, the agent makes predictions under a time-masked information boundary, where only pre-cutoff knowledge and leakage-filtered search evidence are accessible. The raw prediction is then parsed, normalized, compared with the gold answer, and aggregated into evaluation scores at the item, question, and model levels.

3 Method

3.1 Problem Formulation

OracleProto evaluates the native forecasting capability of large language models under a bounded information environment. Each instance is a resolved event whose ground truth is hidden from the model and used only for scoring; the model must select from a finite candidate set using only information that would have been available before resolution. The model’s knowledge cutoff constrains what may be encoded in parameters, and temporal masking constrains what may be retrieved during inference.

Let

$$\mathcal{D} = \{q_i\}_{i=1}^N, \quad q_i = (x_i, \mathcal{A}_i, Y_i, \tau_i, \rho_i), \quad (1)$$

where x_i is the question text, $\mathcal{A}_i$ is the finite candidate set, $Y_i \subseteq \mathcal{A}_i$ is the verified answer, τ_i is the resolution time, and ρ_i specifies the selection structure (single- or multi-answer). The model receives only the time-masked view

$$q_i^{\text{in}} = (x_i, \mathcal{A}_i, \chi_i, \rho_i), \quad (2)$$

where χ_i is the prediction cutoff. For a model M with knowledge cutoff κ_M , the admissible prediction set is

$$\mathcal{D}_M^{\text{pred}} = \{q_i \in \mathcal{D} \mid \kappa_M \leq \chi_i < \tau_i\}, \quad (3)$$

which excludes parametric leakage when $\chi_i < \kappa_M$ and post-resolution information when $\chi_i \geq \tau_i$. Instances violating this condition are logged with their exclusion reason rather than scored as forecasting failures, which is essential for comparing models with different cutoffs. The pipeline composes task construction, bounded inference, and scoring,

$$\mathcal{D} \rightarrow \mathcal{D}_M^{\text{pred}} \rightarrow F_M(q_i^{\text{in}}, \mathcal{I}_{i,M}^{\text{vis}}) \rightarrow \hat{Y}_{i,M} \rightarrow \mathcal{E}_M, \quad (4)$$

with F_M the time-masked forecasting system, $\mathcal{I}_{i,M}^{\text{vis}}$ the information visible to the model, and $\mathcal{E}_M$ the evaluation system. The time-replay assumption that licenses replaying resolved events as forecasting tasks is discussed in Section A.1.

3.2 Forecasting Dataset Framework

Each instance has a finite answer space ($|\mathcal{A}_i| \geq 2$), a verified answer subset $Y_i \subseteq \mathcal{A}_i$, and a resolution time τ_i . Single-answer questions satisfy $|Y_i| = 1$ and multi-answer questions satisfy $|Y_i| \geq 1$; the structural constraint ρ_i determines the valid output cardinality. Given a temporal masking offset δ , the prediction cutoff is

$$\chi_i = \tau_i - \delta, \quad (5)$$

which keeps the visible retrieval environment before the day on which the answer becomes verifiable. To remove surface variation in natural-language outputs, an answer normalization map $\phi_i : \mathcal{A}_i \rightarrow \mathcal{L}_i$ projects predictions and ground truth into a finite label set, so scoring compares $G_i = \phi_i(Y_i)$ and $\hat{G}_{i,M} = \phi_i(\hat{Y}_{i,M})$ as finite sets. Field-format details and the four resulting reproducibility properties are in Section B.1.

3.3 Time-Masked Forecasting System

The forecasting system

$$F_M : (q_i^{\text{in}}, \mathcal{I}_{i,M}^{\text{vis}}) \rightarrow \hat{Y}_{i,M}, \quad \hat{Y}_{i,M} \subseteq \mathcal{A}_i, \quad (6)$$

returns a subset of the candidate answer set; no continuous score or subjective confidence is required. The visible information combines parametric knowledge before κ_M with retrieval results before χ_i ,

$$\mathcal{I}_{i,M}^{\text{vis}} = \mathcal{K}_{\leq \kappa_M}^M \cup \mathcal{T}_{\leq \chi_i}. \quad (7)$$

The model’s action space contains query proposal and answering; the cutoff χ_i is held by the evaluation system and injected at the tool layer, so the model cannot modify the information boundary.

Temporal masking enforces $\mathcal{T}_{\leq \chi_i}$ at two layers. The tool layer executes retrieval with the prediction cutoff,

$$u_{i,t} = \text{Search}(r_{i,t}; \chi_i), \quad (8)$$

through a date-restricted retrieval backend; our implementation uses Tavily, but the framework is backend-agnostic when the boundary is enforced by the evaluator. Returned snippets, cached pages, and aggregate summaries can still carry information after χ_i , so an auxiliary detector H_{aux} performs a content-level audit before results enter the main-model context,

$$\tilde{u}_{i,t} = \{v \in u_{i,t} \mid H_{\text{aux}}(v, \chi_i) = 1\}. \quad (9)$$

H_{aux} sees only the retrieval payload (title, URL, publication time, snippet, cutoff date) and never the question, candidates, ground truth, or main-model dialogue, so it cannot serve as an answer auditor. Detection outcomes, latency, and error types are logged as audit metadata.

Forecasting interaction. For an admissible instance, the deterministic renderer R produces the initial message $m_{i,0} = R(q_i^{\text{in}})$, which contains the question, candidate set, prediction cutoff, selection structure, and the requirement that the final answer be parseable as a discrete prediction. The interaction is bounded by step cap T and search-call cap C . At step t , conditioning only on the previous history, the model selects an action

$$a_{i,t} \in \mathcal{U} \cup \{\text{answer}\}, \quad (10)$$

where $\mathcal{U}$ collects allowed tool actions and the key tool action is time-masked search. Filtered results are appended to the context as $m_{i,t} = m_{i,t-1} \oplus (a_{i,t}, \tilde{u}_{i,t})$ under deterministic concatenation $\oplus$; if a search is requested after the budget is exhausted, no external call is made and a budget-status message is appended so the model must answer within the remaining steps. When the model submits a final answer $o_{i,t}$, the parser

$$\hat{Y}_{i,M} = P(o_{i,t}; q_i^{\text{in}}) \quad (11)$$

checks $\mathcal{A}_i$ and ρ_i ; missing answers, invalid formats, candidate-set mismatches, and structural-constraint violations are marked as invalid outputs, as is the case when no parseable answer is produced within T steps. The full procedure is summarized in Algorithm 1.

3.4 Evaluation System

Evaluation maps raw outputs $o_{i,M}$ through a parser Ψ_i and the normalization ϕ_i ,

$$o_{i,M} \xrightarrow{\Psi_i} \Psi_i(o_{i,M}) \in 2^{\mathcal{A}_i} \cup \{\perp\} \xrightarrow{\phi_i} \hat{G}_{i,M}, \quad (12)$$

Algorithm 1 Time-masked discrete forecasting loop. The cutoff is held by the evaluation system and injected at the tool layer; the evaluated model can observe questions, candidate answers, and filtered retrieval results, but cannot modify the temporal boundary.

Input: Instance $q_i = (x_i, \mathcal{A}_i, Y_i, \tau_i, \rho_i)$, model M , knowledge cutoff κ_M , prediction cutoff χ_i , maximum steps T , maximum search calls C

Output: Parsed discrete prediction or a boundary/invalid-output marker

```

1: if  $\neg(\kappa_M \leq \chi_i < \tau_i)$  then
2:   return outside-boundary marker
3: end if
4:  $q_i^{\text{in}} \leftarrow (x_i, \mathcal{A}_i, \chi_i, \rho_i)$ 
5:  $m_0 \leftarrow R(q_i^{\text{in}})$ 
6:  $c \leftarrow 0$ 
7: for  $t = 1, \dots, T$  do
8:    $a_t \leftarrow M(m_{t-1})$ 
9:   if  $a_t$  is a final answer then
10:    return  $P(a_t; q_i^{\text{in}})$ 
11:   end if
12:   if  $a_t$  is a search action and  $c < C$  then
13:     Let  $r_t$  be the query proposed by  $M$ 
14:      $u_t \leftarrow \text{Search}(r_t; \chi_i)$ 
15:      $\tilde{u}_t \leftarrow \text{AuxLeakFilter}(u_t, \chi_i)$ 
16:      $m_t \leftarrow m_{t-1} \oplus (a_t, \tilde{u}_t)$ 
17:      $c \leftarrow c + 1$ 
18:   else
19:      $m_t \leftarrow m_{t-1} \oplus \text{budget/status message}$ 
20:   end if
21: end for
22: return invalid-output marker

```

$\triangleright$ tool-level date filtering
 $\triangleright$ content-level leakage detection

where $\perp$ marks unevaluable outputs (missing answer, refusal, invalid format, or candidate-set mismatch). Validity and correctness are kept distinct: the evaluability indicator

$$v_{i,M} = \mathbb{I}[\Psi_i(o_{i,M}) \neq \perp] \quad (13)$$

asks whether the output is parseable and is reported as the validity component, while the strict item-level correctness indicator

$$r_{i,M} = \mathbb{I}[\hat{G}_{i,M} = G_i] \quad \text{when } v_{i,M} = 1 \quad (14)$$

asks whether the normalized prediction equals the gold set. The full evaluation system has four levels,

$$\mathcal{E}_M = (\mathcal{E}_M^{\text{valid}}, \mathcal{E}_M^{\text{item}}, \mathcal{E}_M^{\text{question}}, \mathcal{E}_M^{\text{model}}), \quad (15)$$

measuring parseability, single-prediction correctness, repeated-trial behavior on one question, and aggregate behavior on $\mathcal{D}_M^{\text{pred}}$, respectively. If a question is run independently S times, the question-level object $\{\hat{G}_{i,M}^{(s)}\}_{s=1}^S$ captures accuracy, stability, consistency, and coverage across repeated attempts, and model-level evaluation aggregates over $\mathcal{D}_M^{\text{pred}}$,

$$\mathcal{E}_M^{\text{model}} = \Gamma(\{\mathcal{E}_{i,M}^{\text{question}} \mid q_i \in \mathcal{D}_M^{\text{pred}}\}), \quad (16)$$

under a predefined rule Γ that may report accuracy, evaluability rate, stability across repeated trials, consistency across question types, multi-answer handling, or tool-use cost. Multi-answer set-difference relations and the full evaluation chain are in Section A.2.

3.5 Reproducibility and Leakage Boundary

A complete run is the tuple

$$\mathcal{R} = (\mathcal{D}, M, \kappa_M, \delta, T, C, R, \Psi, \phi, \Gamma), \quad (17)$$

where T caps interaction steps, C caps search calls, R is the input renderer, and Γ is the aggregation rule. The detector H_{aux} , its prompt SHA-256, and its failure policy are recorded as run-configuration metadata so the leakage barrier is itself byte-reproducible. Once $\mathcal{R}$ is fixed, sample admission, prompt rendering, tool masking, parsing, and aggregation all have an auditable replay path; per-question stochasticity is captured by recording the sampling settings and the index s of each independent run.

The framework controls three information channels rather than relying on an unverifiable claim that leakage has been excluded. *Parametric knowledge*: the admissibility condition Eq. (3) removes instances that may already be resolved within the model’s training horizon. *Tool-mediated knowledge*: retrieval is restricted by χ_i , and the model can propose queries but cannot alter the cutoff. *Retrieval-result content*: $H_{\text{aux}}(v, \chi_i)$ audits returned text before it enters the main-model context, reducing the risk that snippets, cached pages, or aggregate pages carry future information. Residual leakage risks that this framework cannot eliminate are catalogued in Section A.3.

4 Experiment

4.1 Experimental Setup

4.1.1 Dataset

The evaluation set $\mathcal{D}$ is a curated subset of FutureX-Past [2, 13], retaining $|\mathcal{D}| = 80$ discrete-choice questions whose event-resolution dates τ_q fall between 2026-03-11 and 2026-04-14. The joint distribution over question type and answering mode is shown in Table 1.

Table 1: Question-type distribution of evaluation set $\mathcal{D}$.

Question Type	Answering Mode		
	Single-answer	Multi-answer	Total
Yes/No	37	0	37
Binary Choice	3	0	3
Multiple Choice	32	8	40
Total	72	8	80

Field-level format and the manual zero-leakage audit conducted at construction time are described in Section B.1.

4.1.2 Models Under Test

The six models under test, listed in Table 2, are all served via the OpenAI-compatible POST `/chat/completions` endpoint with the provider-native browsing channel disabled (Section B.2).

Table 2: Models under test. “Excluded by Cutoff” counts the admissibility violations $\{q \in \mathcal{D} : \chi_q < \tau_m^{\text{cut}}\}$ from Eq. (3).

Model m	Training Cutoff τ_m^{cut}	Excluded by Cutoff
DeepSeek-V3.2-Exp	2025-09-29	0
GLM 5	2026-02-11	0
Qwen3.5-Flash	2026-02-25	0
MiniMax M2.5	2026-02-12	0
Kimi K2.5	2026-01-27	0
Doubao Seed 2.0 Lite	2026-03-10	0

Each τ_m^{cut} is identified with κ_M and taken from the official model card or provider disclosure, last accessed on 2026-04-30.³ All $\tau_q \in \mathcal{D}$ fall strictly after every τ_m^{cut} in Table 2, so each model’s effective evaluable set under $\delta = 1$ day is $\mathcal{D}_m^{\text{eval}} = \mathcal{D}$.

4.1.3 Inference and Retrieval Protocol

Each question is sampled $n = 3$ times. The ReAct loop is capped at 6 message rounds and $C = 4$ search calls; the search tool wraps Tavily with $R_{\text{tav}} = 5$ results per call, yielding up to $R_{\text{tav}} \cdot C = 20$ candidate results per question. The temporal cutoff $\chi_q = \tau_q - \delta$ is injected by the implementation and never exposed through the tool interface. Once the search budget is exhausted or the round limit approaches, the loop transitions to forced finalization, requiring `\boxed{\dots}` in the final reply. The probability/belief protocol is disabled in this run, and metrics are computed at the

³When the disclosed cutoff is given at month-level granularity, the last day of the disclosed month is adopted as the most conservative admissibility choice.

discrete letter-set level. LLM call parameters, API-key scheduling, and retry policies are documented in Sections B.3 and B.6; prompt-template hashing and the configuration snapshot are documented in Section B.7.

4.1.4 Anti-Leakage Barriers

Three barriers protect the boundary $\chi_q = \tau_q - \delta$ on top of the dataset’s manual zero-leakage audit: training-cutoff admissibility drops q with $\chi_q < \kappa_M$ at the runner; algorithmic Tavily date restriction filters search results by their published date; and a content-level LLM detector audits each surviving result with a binary keep/drop verdict and fails closed on detector errors. Detector configuration is in Section B.4; the scheme comparison and quantitative residual-leakage audit are in Section B.5 and Section 4.3.4.

4.2 Metric Overview

4.2.1 Notation

Per question $q \in \mathcal{D}^{\text{eval}}$ and trial $j \in \{1, \dots, n\}$, $\hat{S}_{q,j}, G_q \subseteq \mathcal{O}_q$ are the parsed prediction set and gold set, with $k_q = |\mathcal{O}_q|$ and $m_q = |G_q|$, and $c_{q,j} = \mathbb{I}[\hat{S}_{q,j} = G_q]$ is the strict-equality verdict. Trials free of training-cutoff exclusion and transport-layer call errors form $\mathcal{J}_q^{\text{cnt}}$, and the parsable subset $\mathcal{J}_q^{\text{ok}} \subseteq \mathcal{J}_q^{\text{cnt}}$ has size $K_q^{\text{eff}} = |\mathcal{J}_q^{\text{ok}}|$. The scorable sample set is $\mathcal{S} = \{(q, j) : q \in \mathcal{D}^{\text{eval}}, c_{q,j} \neq \text{None}\}$.⁴ The dataset is partitioned by question type into $B = \{\text{yes/no, binary, mc}\}$ with weights (0.15, 0.15, 0.70). The multiple-choice bucket is further sliced by answering mode into MC-single and MC-multi for the per-bucket table, sharing the 0.70 family weight.

4.2.2 Accuracy

The per-trial scoring primitive is exam-style partial credit: any false positive sets the trial score to 0, and otherwise the score is the recall fraction $|TP|/|G_q|$,

$$\text{exam-score}(\hat{S}_{q,j}, G_q) = \begin{cases} \frac{|\hat{S}_{q,j} \cap G_q|}{|G_q|}, & \hat{S}_{q,j} \setminus G_q = \emptyset, \\ 0, & \hat{S}_{q,j} \setminus G_q \neq \emptyset, \end{cases} \quad (18)$$

which reduces to strict equality on single-answer questions ($m_q = 1$). The per-bucket exam mean $\text{exam}_{\text{avg}}^{(b)}$ averages Eq. (18) first over $\mathcal{J}_q^{\text{cnt}}$ within a question and then uniformly over $q \in \mathcal{D}_b^{\text{eval}}$, and composite accuracy is the bucket-weighted mean over valid buckets $B_{\text{valid}}(m) = \{b \in B : \text{exam}_{\text{avg}}^{(b),m} \neq \text{None}\}$,

$$\text{Composite Accuracy}_m = \frac{\sum_{b \in B_{\text{valid}}(m)} w_b \text{exam}_{\text{avg}}^{(b),m}}{\sum_{b \in B_{\text{valid}}(m)} w_b}. \quad (19)$$

The strict-equality companion $\text{pass}@1_{\text{avg}}$ takes the intra-question mean of $c_{q,j}$ over $\mathcal{J}_q^{\text{cnt}}$ followed by an equal-weight question average,

$$\text{pass}@1_{\text{avg}} = \frac{1}{|\mathcal{D}^{\text{scored}}|} \sum_{q \in \mathcal{D}^{\text{scored}}} \frac{1}{|\mathcal{J}_q^{\text{cnt}}|} \sum_{j \in \mathcal{J}_q^{\text{cnt}}} c_{q,j}, \quad (20)$$

with $\text{pass}^{\text{any}}@n$ and $\text{pass}^{\text{all}}@n$ replacing the intra-question mean by $\bigvee_j c_{q,j}$ and $\prod_j c_{q,j}$, respectively; the chain $\text{pass}^{\text{all}}@n \leq \text{pass}@1_{\text{avg}} \leq \text{pass}^{\text{any}}@n$ holds by construction.

4.2.3 Consistency

Cohen’s κ chance-corrects strict accuracy against a question-type-conditional baseline,

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad p_{e,q} = \begin{cases} 1/k_q, & q \text{ single-answer,} \\ 0.5, & q \text{ multi-answer (per-label coin-flip),} \end{cases} \quad (21)$$

where p_o is the trial-weighted hit rate over $\mathcal{S}$ and p_e is the sample-weighted average of $p_{e,q}$. Fleiss’ κ generalizes the same chance correction to $n \geq 2$ raters per stratum,

$$\kappa_{\text{Fleiss}} = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}, \quad (22)$$

⁴Parse failures score 0 but do not enter Fleiss’ vote tally; the handling of missing buckets and empty denominators is described in Section B.6.

stratifying single-answer questions by k_q and reducing multi-answer questions to per-candidate-letter binary κ before label averaging.

4.2.4 Skill, Cost, and Leakage

The Format Skill Score is a Tversky-similarity-based [11], chance-corrected skill score with $(\alpha, \beta) = (2.0, 0.5)$, penalizing false positives $4\times$ as heavily as false negatives,

$$\text{fss} = \frac{1}{|\mathcal{D}^{\text{ok}}|} \sum_{q \in \mathcal{D}^{\text{ok}}} \frac{\bar{T}_q - T_q^{\text{chance}}}{1 - T_q^{\text{chance}}}, \quad \bar{T}_q = \frac{1}{K_q^{\text{eff}}} \sum_{j \in \mathcal{J}_q^{\text{ok}}} \frac{|P_{q,j} \cap G_q|}{|P_{q,j} \cap G_q| + \alpha |P_{q,j} \setminus G_q| + \beta |G_q \setminus P_{q,j}|}, \quad (23)$$

with T_q^{chance} the closed-form expectation under a question-type-conditional reference distribution. The per-correct-prediction cost amortizes the OpenRouter platform invoice over the model’s difficulty-weighted notional correct-sample count,

$$C_m^{\text{per-correct}} = \frac{C_m^{\text{total}}}{|\mathcal{D}^{\text{eval}}| \cdot n \cdot \text{Composite Accuracy}_m}. \quad (24)$$

The residual leakage rate is reported as the per-audit-item rate FN/N with Wilson 95% CI in Section 4.3.4, and as the leak-conditional pass-through rate $1 - \text{recall}$ in Section B.5. Probabilistic, belief-trace, and behavioral diagnostics are catalogued in Section B.8.

4.3 Experimental Data and Analysis

4.3.1 Overall Performance and Cost

Table 3: Overall performance and cost. Composite accuracy follows Eq. (19); per-correct cost follows Eq. (24).

Model	Composite Accuracy	Total Cost (USD)	Per-Correct Cost (USD)
DeepSeek-V3.2-Exp	0.6016	3.60	0.025
GLM 5	0.6002	7.06	0.048
Qwen3.5-Flash	0.5896	0.45	0.003
MiniMax M2.5	0.5494	3.21	0.024
Kimi K2.5	0.5800	6.79	0.049
Doubao Seed 2.0 Lite	0.5858	0.89	0.006

Analysis. Table 3 reports composite accuracy alongside total and per-correct cost. Composite accuracy spans only 5.2 pp across the six models, with the top five within 2.2 pp, while per-correct cost varies by $16\times$ from Qwen (\$0.003) to Kimi/GLM 5 ($\sim \0.049). Qwen3.5-Flash and DeepSeek-V3.2-Exp jointly define the cost–quality Pareto frontier, with the other four models dominated by at least one of these two: GLM 5 and Kimi pay a $16\times$ per-correct premium over Qwen for composite-accuracy gains of only +1.1 pp and -1.0 pp, respectively.

4.3.2 Per-Question-Type Slice

Table 4: Per-bucket exam mean $\text{exam}_{\text{avg}}^{(b)}$. MC-Single and MC-Multi are answer-mode slices of the multiple-choice bucket and share the 0.70 family weight in the composite.

Model	Yes/No	Binary	MC Single	MC Multi
DeepSeek-V3.2-Exp	0.6261	0.8889	0.5938	0.2986
GLM 5	0.6216	1.0000	0.5729	0.2581
Qwen3.5-Flash	0.6036	0.8889	0.5833	0.2789
MiniMax M2.5	0.6126	1.0000	0.5000	0.1968
Kimi K2.5	0.6036	0.8889	0.5885	0.1898
Doubao Seed 2.0 Lite	0.4828	1.0000	0.6061	0.2460

Analysis. Table 4 reports each model’s exam mean on the four buckets. Doubao leads MC-Single at 0.6061 but trails Yes/No at 0.4828, exhibiting the largest cross-bucket variance, while MC-Multi is the weakest bucket for all six models, with values confined to $[0.19, 0.30]$. DeepSeek’s composite advantage stems from narrow leads on Yes/No and MC-Multi, and Kimi’s MC-Multi at 0.1898, last in the column, explains its drop from 3rd on strict $\text{pass}@1_{\text{avg}}$ to 5th on composite once the multi-choice family is up-weighted to 0.70.

4.3.3 Consistency and Sampling Stability

Table 5: Per-question accuracy, consistency, and skill score. Column definitions follow Eqs. (20) to (23); $\text{pass}^{\text{any}}@n$ and $\text{pass}^{\text{all}}@n$ replace the intra-question average in Eq. (20) by the OR and AND of $c_{q,j}$, respectively.

Model	$\text{pass}@1_{\text{avg}}$	$\text{pass}^{\text{any}}@n$	$\text{pass}^{\text{all}}@n$	Cohen’s κ	Fleiss’ κ	fss
DeepSeek-V3.2-Exp	0.5756	0.8000	0.3500	0.3101	0.3452	0.3758
GLM 5	0.5696	0.7625	0.3625	0.2985	0.4289	0.3627
Qwen3.5-Flash	0.5565	0.7500	0.3875	0.2784	0.4515	0.3433
MiniMax M2.5	0.5314	0.6875	0.3250	0.2391	0.3867	0.2997
Kimi K2.5	0.5612	0.8000	0.3000	0.2852	0.2975	0.3315
Doubao Seed 2.0 Lite	0.5056	0.7000	0.2833	0.1846	0.4203	0.2349

Analysis. The strict $\text{pass}@1_{\text{avg}}$ ranking matches Cohen’s κ exactly, since chance correction is a monotone rescaling. DeepSeek and Kimi tie for first on $\text{pass}^{\text{any}}@n$ at 0.80, while Qwen leads $\text{pass}^{\text{all}}@n$ at 0.3875 and Fleiss’ κ at 0.4515; Kimi’s $\text{pass}^{\text{any}}@n - \text{pass}^{\text{all}}@n = 0.50$ is the largest of the six and signals high-variance answers, whereas Qwen’s answers are the most consistent across trials. Qwen overtakes Kimi on fss because the $(\alpha, \beta) = (2.0, 0.5)$ Tversky setting penalizes false positives $4\times$ more heavily than false negatives, and Qwen’s MC-Multi selections contain fewer letters on average.

4.3.4 Leakage Rate Audit

The accuracy claims rest on the premise that the model never accesses information dated later than $\chi_q = \tau_q - \delta$. To estimate the actual residual leakage rate, three of the six tested models (Kimi K2.5, Qwen3.5-Flash, GLM 5) were each evaluated on 30 questions across 3 independent runs, and one search result was sampled uniformly at random from each test item’s up to $R_{\text{tav}} \cdot C = 20$ candidates, totaling $N = 270$ verdicts. Detector verdicts were produced by an independent Qwen3.5-Flash instance with the detector-specific prompt; two authors produced the human labels and cross-validated each other’s annotations. The confusion matrix is in Table 6.

Table 6: Manual audit confusion matrix. TP/TN: detector and human agree (leak/no-leak); FN: real leak missed by the detector, reaching the main LLM; FP: non-leak erroneously dropped.

Category	TP	TN	FP	FN	Total
Count	235	31	1	3	270
Proportion	87.03%	11.48%	0.37%	1.11%	100%

Analysis. Recall is $235/238 \approx 98.7\%$ and specificity is $31/32 \approx 96.9\%$. The per-audit-item residual leakage rate $\text{FN}/N = 3/270 \approx 1.1\%$ has Wilson 95% upper bound $\approx 3.2\%$, an order of magnitude below the no-detector baseline and comparable to the lower end of the Tavily-only band; the leak-conditional companion $3/238 \approx 1.3\%$ approaches the manual-annotation floor at marginal cost two orders of magnitude lower (Section B.5).

5 Discussion

Forecasting as a trainable capability. OracleProto’s current instantiation targets evaluation, yet each row of the dataset already carries the signal a trainer would need: a retrieval trace, a reasoning trajectory, and a final answer that together form a complete training pair for supervised fine-tuning and reinforcement learning, without any change to the dataset contract. If forecasting datasets can be produced and accumulated as text corpora are, forecasting capability shifts from an incidental emergent behavior of LLMs to a model-native skill that admits systematic training and continuous optimization.

A monotonically growing forecasting corpus. The 80 questions used in this paper are a first instantiation of the framework; the structural observation behind it is that globally accumulating expired forecasting benchmarks, such as FutureX-Past, expired ForecastBench items, and Metaculus historical records, are no longer contaminated waste under this framework but training corpora that can be reactivated as zero-contamination forecasting samples whenever they fall under a model’s knowledge cutoff. The set of usable forecasting samples grows monotonically over time, giving forecasting evaluation and training a data-flywheel structure analogous to natural-language pretraining.

From information boundaries to auditable decision support. The information-boundary mechanism serves more than evaluation fairness: it offers a reference framework for the auditable deployment of LLMs in high-stakes decision settings. When an LLM is asked to deliver a judgment in finance, policy, or public-safety contexts, “on what information did it base this judgment” is a core compliance question; the run record and information-boundary audit produced by OracleProto provide one technical path to answering it.

6 Conclusion

OracleProto recasts LLM-native forecasting evaluation from a one-off live run into a reproducible, auditable dataset-level task. By jointly enforcing knowledge-cutoff admission, tool-layer temporal masking, and content-level leakage detection, resolved events regain forecasting validity within a discrete answer space. In the 80-question instantiation drawn from FutureX-Past, the framework simultaneously characterizes the forecasting quality, sampling stability, and cost efficiency of six contemporary LLMs while holding the residual leakage rate near 1%. We hope OracleProto turns the dataset itself into the central object of LLM forecasting evaluation, supplying a cumulative digital asset for evaluation, training, and forecasting-agent research.

References

- [1] N. Chandak, S. Goel, A. Prabhu, M. Hardt, and J. Geiping. Scaling open-ended reasoning to predict the future, 2026. URL <https://arxiv.org/abs/2512.25070>.
- [2] futurex-ai. FutureX-Past Dataset. Hugging Face Datasets, 2025. URL <https://huggingface.co/datasets/futurex-ai/Futurex-Past>.
- [3] D. Halawi, F. Zhang, C. Yueh-Han, and J. Steinhardt. Approaching Human-Level Forecasting with Language Models, 2024. URL <https://arxiv.org/abs/2402.18563>.
- [4] E. Karger, H. Bastani, C. Yueh-Han, Z. Jacobs, D. Halawi, F. Zhang, and P. E. Tetlock. ForecastBench: A dynamic benchmark of AI forecasting capabilities, 2025. URL <https://arxiv.org/abs/2409.19839>.
- [5] Z. Li, Y. Wang, A. E. Lahib, Y.-J. Xia, and X. Pi. Simulated ignorance fails: A systematic study of LLM behaviors on forecasting problems before model knowledge cutoff, 2026. URL <https://arxiv.org/abs/2601.13717>.
- [6] Z. Liu, P. Han, H. Yu, H. Li, and J. You. Time-R1: Towards comprehensive temporal reasoning in LLMs, 2025. URL <https://arxiv.org/abs/2505.13508>.
- [7] Metaculus. Metaculus FAQ. Online documentation, 2026. URL <https://www.metaculus.com/faq/>. Accessed: 2026-05-05.
- [8] K. Murphy. Agentic forecasting using sequential bayesian updating of linguistic beliefs, 2026. URL <https://arxiv.org/abs/2604.18576>.
- [9] D. Paleka, S. Goel, J. Geiping, and F. Tramèr. Pitfalls in evaluating language model forecasters, 2025. URL <https://arxiv.org/abs/2506.00723>.
- [10] B. Turtel, D. Franklin, K. Skotheim, L. Hewitt, and P. Schoenegger. Outcome-based reinforcement learning to predict the future, 2025. URL <https://arxiv.org/abs/2505.17989>.
- [11] A. Tversky. Features of similarity. *Psychological Review*, 84(4):327–352, 1977. doi: 10.1037/0033-295X.84.4.327.
- [12] C. Ye, Z. Hu, Y. Deng, Z. Huang, M. D. Ma, Y. Zhu, and W. Wang. MIRAI: Evaluating LLM agents for event forecasting, 2024. URL <https://arxiv.org/abs/2407.01231>.
- [13] Z. Zeng, J. Liu, S. Chen, T. He, Y. Liao, Y. Tian, J. Wang, Z. Wang, Y. Yang, L. Yin, M. Yin, Z. Zhu, T. Cai, Z. Chen, J. Chen, Y. Du, X. Gao, J. Guo, L. Hu, J. Jiao, X. Li, J. Liu, S. Ni, Z. Wen, G. Zhang, K. Zhang, X. Zhou, J. Blanchet, X. Qiu, M. Wang, and W. Huang. FutureX: An advanced live benchmark for LLM agents in future prediction, 2025. URL <https://arxiv.org/abs/2508.11987>.

A Method Details

A.1 Time-replay assumption

A resolved event can serve as a forecasting task whenever its temporal boundary recreates the information state before resolution. The condition admits an event $q_i = (x_i, \mathcal{A}_i, Y_i, \tau_i, \rho_i)$ when an evaluator can place the model at χ_i with parametric knowledge bounded by κ_M and external retrieval bounded by χ_i . Three classes of events fall outside this admission and no temporal mask can rescue them: (i) events whose source-corpus framing already presupposes the resolution, such as post-resolution recaps or anniversary articles; (ii) events whose answer was decided before χ_i but reported only after τ_i , where the question is answerable from pre-cutoff evidence and the dataset only delays its discovery; (iii) items whose stem text x_i contains explicit cues to the post- χ_i outcome, against which only the construction-time manual audit in Section B.1 can defend.

Conventional factual question answering would treat such an event as (x_i, Y_i) and ask the model to recover Y_i from x_i . OracleProto rewrites the four-tuple at the dataset level into the prediction-time object

$$(x_i, \mathcal{A}_i, Y_i, \tau_i, \rho_i) \implies q_i^{\text{in}} = (x_i, \mathcal{A}_i, \chi_i, \rho_i), \quad (25)$$

retaining Y_i only for scoring. The visible information set follows from the rewrite,

$$\mathcal{I}_{i,M}^{\text{vis}} = \mathcal{K}_{\leq \kappa_M}^M \cup \mathcal{T}_{\leq \chi_i}, \quad \mathcal{T}_{\leq \chi_i} = \{u \mid \text{time}(u) \leq \chi_i\}, \quad (26)$$

so the model sees parametric knowledge no younger than κ_M and tool-mediated knowledge no younger than χ_i . The pipeline that materialises Eqs. (25) and (26) into a concrete corpus, the on-disk field format, and the construction-time audit that protects the third exclusion class are detailed in Section B.1.

A.2 Evaluation chain details

The parser Ψ_i extracts the last `\boxed{\dots}` occurrence in the assistant’s terminal message and dispatches by $q_i.\text{question_type}$. Algorithm 2 states the rules in pseudocode.

The unevaluable marker $\perp$ absorbs missing answers, refusals, malformed payloads, candidate-set mismatches, and structural-constraint violations under one symbol. Validity and correctness are reported separately,

$$v_{i,M} = \mathbb{I}[\Psi_i(o_{i,M}) \neq \perp], \quad r_{i,M} = \mathbb{I}[\hat{G}_{i,M} = G_i] \text{ when } v_{i,M} = 1, \quad (27)$$

so a high parse-failure rate does not silently inflate the error rate. Multi-answer set comparisons additionally support

$$\text{TP}_{i,M} = \hat{G}_{i,M} \cap G_i, \quad \text{FP}_{i,M} = \hat{G}_{i,M} \setminus G_i, \quad \text{FN}_{i,M} = G_i \setminus \hat{G}_{i,M}, \quad (28)$$

which feed both the exam-style score in Eq. (18) and the Tversky aggregator in Eq. (23).

The evaluation chain stratifies into four levels, each with its own input object and aggregator. Table 7 summarises the contracts.

Table 7: Evaluation chain $\mathcal{E}_M = (\mathcal{E}_M^{\text{valid}}, \mathcal{E}_M^{\text{item}}, \mathcal{E}_M^{\text{question}}, \mathcal{E}_M^{\text{model}})$. The output column lists representative scalars; many alternatives share the same input object.

Level	Input object	Aggregator	Representative output
$\mathcal{E}_M^{\text{valid}}$	$v_{i,M}, \Psi_i(o_{i,M})$	per-question and trial mean	evaluability rate, parse-failure rate, error rate
$\mathcal{E}_M^{\text{item}}$	$\hat{G}_{i,M}, G_i$	$r_{i,M}$, exam score, Tversky $T_{i,M}$	item correctness, partial credit
$\mathcal{E}_M^{\text{question}}$	$\{\hat{G}_{i,M}^{(s)}\}_{s=1}^S, K_q^{\text{eff}}$	per-question mean, OR, AND, majority vote	$\text{pass}@1_{\text{avg}}$, $\text{pass}^{\text{any}}@n$, $\text{pass}^{\text{all}}@n$, MV-Acc, FSS _q
$\mathcal{E}_M^{\text{model}}$	$\{\mathcal{E}_{i,M}^{\text{question}}\}_{q_i \in \mathcal{D}_M^{\text{pred}}}$	Γ from Eq. (19)	composite accuracy, FSS, Cohen κ , BI, per-correct cost

The s -th independent run on the same question is written

$$\hat{Y}_{i,M}^{(s)} = F_M^{(s)}(q_i^{\text{in}}; \mathcal{I}_{i,M}^{\text{vis}}), \quad (29)$$

and the chain

$$o_{i,M} \rightarrow \Psi_i(o_{i,M}) \rightarrow \hat{G}_{i,M} \rightarrow \mathcal{E}_{i,M}^{\text{item}} \rightarrow \mathcal{E}_{i,M}^{\text{question}} \rightarrow \mathcal{E}_M^{\text{model}} \quad (30)$$

Algorithm 2 Parser dispatch $\Psi_i : o \mapsto \widehat{G}_{i,M} \cup \{\perp\}$, implemented in `forecast_eval.parser.parse_answer`. Each question type has its own dispatch routine; structural defects collapse into the unevaluable marker $\perp$.

Input: Raw assistant output o ; question record $q_i = (x_i, \mathcal{A}_i, Y_i, \tau_i, \rho_i)$

Output: Letter set $\widehat{G}_{i,M} \subseteq \mathcal{A}_i$ on success, otherwise the marker $\perp$

```

1: function PARSEANSWER( $o, q_i$ )
2:    $p \leftarrow$  payload of the last \boxed{...} match in  $o$   $\triangleright$  strip surrounding whitespace
3:   if  $p$  is empty then
4:     return  $\perp$ 
5:   end if
6:   if  $q_i$ .question_type = yes_no then
7:     return PARSEYESNO( $p$ )
8:   else if  $q_i$ .question_type = binary_named then
9:     return PARSEBINARYNAMED( $p, q_i$ .options)
10:  else if  $q_i$ .question_type = multiple_choice then
11:    return PARSEMULTIPLECHOICE( $p, q_i$ .options)
12:  end if
13:  return  $\perp$   $\triangleright$  unrecognised question type
14: end function

15: function PARSEYESNO( $p$ )
16:   if lower( $p$ ) = "yes" then
17:     return  $\{A\}$ 
18:   else if lower( $p$ ) = "no" then
19:     return  $\{B\}$ 
20:   end if
21:   return  $\perp$ 
22: end function

23: function PARSEBINARYNAMED( $p, options$ )
24:   for each option  $\ell$  in options at index  $i_\ell$  do
25:     if lower(strip( $\ell$ )) = lower( $p$ ) then
26:       return  $\{\text{letter}(i_\ell)\}$ 
27:     end if
28:   end for
29:   return  $\perp$ 
30: end function

31: function PARSEMULTIPLECHOICE( $p, options$ )
32:    $T \leftarrow$  tokens of  $p$  split on whitespace or comma
33:   if  $T$  is empty then
34:     return  $\perp$ 
35:   end if
36:   for each token  $t \in T$  do
37:     if  $|t| \neq 1$  or letter_index( $t$ )  $\notin [0, |options|)$  then
38:       return  $\perp$   $\triangleright$  out-of-range or multi-character token
39:     end if
40:   end for
41:   return frozenset( $T$ )
42: end function

```

keeps every metric computed from the same normalised discrete answer space. The question-level object retains the per-trial Tversky score, the parser verdict, and the boxed letter set, so Γ instantiates as `pass@lavg`, `passany@n`, `passall@n`, `examavg`, FSS, Cohen κ , Fleiss κ , Brier index, NLL, MBS, ABI, or per-correct cost without re-deriving any sample-level quantity.

Chance correction admits closed forms that respect the question structure. Table 8 summarises the per-question reference rates used by Cohen κ , Fleiss κ , and FSS.

The Cohen κ baseline uses $p_{e,q} = 1/k_q$ for single-answer questions; multi-answer questions use a per-label coin-flip $p_{e,q} = 0.5$, since the strict 0.5^{k_q} baseline collapses for large k_q and would inflate κ to within rounding distance of the raw accuracy. Fleiss κ stratifies single-answer questions by k_q , computes the textbook formula per stratum, then aggregates by question count, since the marginal category proportions are only well-defined inside a fixed category

Table 8: Per-question chance baselines used by the discrete-native family.

Metric	Single-answer baseline	Multi-answer baseline
Cohen $\kappa(p_{e,q})$	$1/k_q$ (uniform letter pick)	0.5 per label, taken as the sample-weighted mean across labels
Fleiss κ	textbook formula stratified by k_q , then aggregated by question count	per-candidate-letter binary κ averaged across labels
FSS chance baseline T_q^{chance}	$1/k_q$	closed-form $\mathbb{E}[\text{Tversky}]$ in Eq. (31)

space; multi-answer questions reduce to a per-candidate-letter binary κ before label averaging. The Tversky baseline for multi-answer questions has the closed-form expectation

$$T_q^{\text{chance}} = \mathbb{E}[\text{Tversky}] = \sum_{tp=1}^{m_q} \sum_{fp=0}^{k_q-m_q} \binom{m_q}{tp} 2^{-m_q} \binom{k_q-m_q}{fp} 2^{-(k_q-m_q)} \frac{tp}{tp + \alpha fp + \beta(m_q - tp)} \quad (31)$$

under a uniform per-label 0.5 reference distribution, which evaluates in $O(m_q(k_q - m_q))$ time. The Brier index averages per-question scores before taking the square root,

$$\text{BI} = 100(1 - \sqrt{\overline{\text{BS}}}), \quad \overline{\text{BS}} = \frac{1}{|\mathcal{D}^{\text{ok}}|} \sum_q \text{BS}_q, \quad (32)$$

so the metric remains monotone in question difficulty, and the adjusted Brier index uses a sign-symmetric convention

$$\text{ABI} = \begin{cases} 100(1 - \sqrt{\overline{\text{ABS}}}), & \overline{\text{ABS}} \geq 0, \\ 100(1 + \sqrt{|\overline{\text{ABS}}|}), & \overline{\text{ABS}} < 0, \end{cases} \quad \text{ABS}_q = \text{BS}_q - \gamma_q, \quad (33)$$

so that a model beating the per-question baseline γ_q raises the score, and the curve is continuous at $\overline{\text{ABS}} = 0$.

Model-level evaluation aggregates over $\mathcal{D}_M^{\text{pred}}$,

$$\mathcal{E}_M^{\text{model}} = \Gamma(\{\mathcal{E}_{i,M}^{\text{question}} \mid q_i \in \mathcal{D}_M^{\text{pred}}\}), \quad (34)$$

optionally weighted by question-type bucket. The composite-by-subtype rule in Eq. (19) drops buckets whose slice value is None and renormalises the remaining weights, so the composite is well-defined whenever at least one bucket contributes. The same rule covers metrics that are undefined on a given bucket by construction, such as the multi-only Hamming score on the yes/no bucket: the slice returns None and the bucket is dropped, instead of inflating the global average with a degenerate zero.

A.3 Residual leakage risks

Five risks remain after the four-layer barrier and the dataset-level audit, each tied to a different channel through which post-cutoff information can re-enter the visible set $\mathcal{T}_{i,M}^{\text{vis}}$.

Parametric coverage. The true content of a model’s training corpus is rarely verifiable from outside, and the disclosed knowledge cutoff is at most a proxy for what the model knows. The framework absorbs this proxy into the admissibility condition $\kappa_M \leq \chi_i < \tau_i$, conservatively rounded to the last day of the disclosed month when the cutoff is given at month-level granularity, so any item whose resolution date falls inside the disclosed month is excluded by default. Cross-model triangulation through paired runs is the only further safeguard available without insider access: a question that all six models in Table 2 answer identically on the eligible side is downweighted in subjective reading even when all individual admissions pass.

Metadata accuracy. Tavily’s date filter operates on `published_date`; pages with missing or wrong publication metadata may slip past the algorithmic layer. The content-level detector then catches the body when it explicitly references post-cutoff events, but cannot recover from cases where the body itself is silent on time. The detector therefore acts as a second line, not a substitute for accurate metadata; Table 17 maps the residual failure modes.

Republication. Cached pages, mirrors, and aggregator listings can pin an old timestamp on a body that was silently updated. Only the body wording reaches the detector for adjudication, so a mirror that copies an old page header but quotes a newer fact-check still drops at body level. Dynamically-generated landing pages whose body contains a single editorial frame and no datable claim remain the residual case for this channel.

Algorithm 3 Dataset construction pipeline, executed by `scripts.build_dataset.py` once per corpus. Every step is deterministic given the upstream input, so re-running the pipeline on a byte-identical source produces a byte-identical SQLite file.

Input: HuggingFace repository `futurex-ai/Futurex-Past` [2]

Output: SQLite file at `SOURCE_DB` populated with three tables: `questions`, `prompt_templates`, `dataset_metadata`

- 1: ingest the source repository row by row $\triangleright$ *streaming load, no full-corpus buffer*
- 2: retain rows with finite $\mathcal{A}_i$, $|\mathcal{A}_i| \geq 2$, and ISO 8601-parseable τ_i $\triangleright$ *admissibility filter*
- 3: collapse each row to $(x_i, \mathcal{A}_i, Y_i, \tau_i)$ with $\rho_i \in \{\text{single}, \text{multi}\}$ $\triangleright$ *canonical four-tuple*
- 4: normalise $\mathcal{A}_i$ to a JSON array and encode Y_i as a comma-separated letter string $\triangleright$ *e.g. "A" or "A, B"*
- 5: run the construction-time audit described in Section B.1, rejecting or rewriting flagged items $\triangleright$ *manual zero-leakage gate*
- 6: write surviving rows into `questions` (Table 9) and renderer rules into `prompt_templates` (Table 10) $\triangleright$ *persistence*
- 7: `source_db_hash` $\leftarrow$ `sha256(SOURCE_DB)`; `metadata_hash` $\leftarrow$ `sha256(canonical_features_json)` $\triangleright$ *fingerprint*

Question-side cues. The stem text x_i may itself encode subtle hints of the resolution: a phrasing peculiar to post-resolution recaps, an option ordering that differs from the pre-resolution canonical sequence, or a numeric range that already excludes the true outcome. The construction-time manual audit in Section B.1 is the only line of defence against this class, and its precision is bounded by annotator effort; the 80 items in Table 1 pass with two annotators and cross-validation, but a future automated stem-side rewriter that strips post-hoc tense would tighten the bound.

Provider-side hidden retrieval. Some inference endpoints expose internal caches, retrieval middleware, or online-tool channels that the OpenAI-compatible `POST /chat/completions` contract cannot disable. A model whose extra capabilities cannot be turned off must be marked as unsuitable for strict temporal-masking evaluation, since any single hidden hop reopens the boundary; `Settings._post_validate` therefore rejects any model slug ending in `:online`, and the same rule applies to the detector slug under Table 11.

The discrete answer space, model-dependent sample admission, tool-level temporal masking, content-level leakage filtering, deterministic parsing, and hierarchical evaluation reduce these five risks to a residual rate measured directly in Section 4.3.4. The residual rate is reported as the per-audit-item frequency FN/N alongside the leak-conditional pass-through $\text{FN}/(\text{TP} + \text{FN})$, so the next benchmark instantiation can compare against a calibrated lower bound rather than against a single point estimate.

B Implementation Details

B.1 Dataset construction and field format

The dataset $\mathcal{D}$ is materialised once at construction time, before any LLM call, so the same corpus replays under different model panels and disclosed cutoffs. Items flow through the pipeline in Algorithm 3, where every step is deterministic given the upstream input, and end as rows in the seven-column `questions` table mapped to the four-tuple q_i in Table 9.

Table 9: Mapping from the four-tuple q_i to the seven-column `questions` table consumed by the renderer in `loader.sync_questions`.

Symbol / role	Column	Content
$q_i.\text{id}$	<code>id</code>	primary key, used as join key against <code>run_results</code>
ρ_i choice cardinality	<code>choice_type</code>	<code>single</code> or <code>multi</code> ; drives parser branch and chance baseline
ρ_i surface form	<code>question_type</code>	<code>yes_no</code> , <code>binary_named</code> , or <code>multiple_choice</code> ; selects the prompt-template family
x_i	<code>event</code>	event description, free text, fed verbatim into the renderer
$\mathcal{A}_i$	<code>options</code>	JSON array of candidate labels in canonical order
Y_i	<code>answer</code>	comma-separated letter string, hidden from the model and used only by scoring
τ_i	<code>end_time</code>	resolution date as ISO 8601, drives $\chi_i = \tau_i - \delta$

The renderer reads only the columns in Table 9 plus the prompt-template family stored in the `prompt_templates` table, itself flattened from `dataset_metadata.features_json.prompt_reconstruction` so the dataset and the rendering rule travel as one byte-stable artefact. Table 10 lists the eight keys the renderer requires; the rendered prompt is a deterministic function of the data record, the renderer R , and χ_i .

Table 10: Required keys in the `prompt_templates` table; each is loaded by `loader.sync_prompt_templates` and consumed by `prompts.render_user_prompt`.

Key	Role
<code>agent_role</code>	system-style header injected into the rendered user message
<code>guidance</code>	end-of-prompt instruction block applied to every question
<code>prompt_template</code>	top-level Python format string composing event, options, output format, and guidance
<code>outcomes_block_rule</code>	specification for rendering the candidate-letter list, audited via <code>tests/test_prompts.py</code>
<code>yes_no_output_format</code>	“yes”/“no” response specification for $\rho_i = \text{single}$ on the yes/no family
<code>binary_named_output_format</code>	label-equality response specification with placeholders <code><options[0]></code> and <code><options[1]></code>
<code>multiple_choice_single_output_format</code>	boxed single-letter specification for the MC single-answer branch
<code>multiple_choice_multi_output_format</code>	boxed letter-set specification for the MC multi-answer branch

Construction-time audit. The audit invoked at step 5 of Algorithm 3 catches three failure modes that no temporal mask can recover from at evaluation time. The first is explicit post-resolution wording in the question stem, which two annotators flag by lexical convention such as past-tense framing or phrases like “the result was”. The second is candidate options that already encode the outcome, which the same two annotators flag by inspecting $\mathcal{A}_i$ against Y_i . The third is multi-answer items whose verified answer is empty, which is enforced both at curation and at parser time via $|G_i| \geq 1$ and rejected before the row enters questions. The two annotators cross-validate every flagged item, with mismatches resolved by rewriting the stem to a pre-resolution form or by removing the item from $\mathcal{D}$.

Resulting properties. Algorithm 3 together with Eqs. (25) and (26) equip $\mathcal{D}$ with four properties on which downstream evaluation depends. Temporal reproducibility holds because the rendered prompt is a deterministic function of the data record, the renderer R , and χ_i , with the renderer fingerprinted via `prompt_templates_hash` and the data record via `source_db_hash`, both written into `run_meta` per evaluation. Model-dependent admissibility enforces $\kappa_M \leq \chi_i < \tau_i$ from Eq. (3) upstream of any LLM call inside `runner.build_task_plan`, logging samples that violate either bound as `error="skipped_training_cutoff"` and keeping them off the inference path. Discrete scorability follows from persisting the parser-normalised letter set $\hat{G}_{i,M}$ alongside the raw output in the per-trial columns `s{j}_final_answer_letters` and `s{j}_final_answer_raw`, which keeps set comparisons in $\mathcal{L}_i$ exact and auditable. Replay across calendar years lets the same corpus run under different M and κ_M without any item-level edit, with comparability bounded only by the disclosed-cutoff drift discussed in Section A.3.

B.2 Routing and provider-side browsing

Model routing is restricted to the OpenAI-compatible `POST /chat/completions` endpoint; no model slug may carry the `:online` suffix, and `Settings._post_validate` raises immediately on violation. The same constraint applies to the detector slug, audited again at detector send-time by `leak_filter._assert_detector_safe` before the request leaves the harness. Table 11 stacks the three send-time barriers.

Table 11: Send-time browsing-barrier assertions; each runs unconditionally on every outbound LLM call so partial configuration drift cannot bypass the boundary.

Assertion site	Rejected on every call
<code>Settings._post_validate</code>	startup: any model slug ending in <code>:online</code> ; missing or placeholder API keys; <code>LEAK_DETECTOR_MODEL</code> ending in <code>:online</code> when <code>ENABLE_SEARCH_LEAK_FILTER=true</code>
<code>llm._assert_no_browsing</code>	per main-LLM request: model ending in <code>:online</code> ; tool list of length $\neq 0$ or 1; tool name $\neq \text{web_search}$; <code>plugins</code> field present in the request body
<code>leak_filter._assert_detector_safe</code>	per detector request: model ending in <code>:online</code> ; any of <code>tools</code> , <code>plugins</code> , <code>tool_choice</code> present in the kwargs

The `web_search` schema in `tools.WEB_SEARCH_SCHEMA` is fixed at module-import time and protected by a module-level assert pinning the only exposed argument and the required field list,

```
parameters.properties.keys() = { query },
parameters.required = [ query ].
```

Any drift that adds an LLM-controllable argument such as `end_date` would break the information barrier and is rejected before any test runs. The cutoff date χ_q is therefore computed inside the harness, never sent through any LLM-visible channel, and is not modifiable by tool-call arguments.

B.3 Inference protocol and search-tool details

Table 12 lists the parameters used in the main run.

Table 12: Inference, ReAct loop, and search-tool parameters.

Parameter	Value
Sampling temperature	0.7
<code>top-p</code>	1.0
<code>max-tokens</code>	12,000
Single-call timeout	240 s
Reasoning-class slug pattern	substring of {o1, o3, o4, r1, qwq}
ReAct message-round cap T	12
Search-call cap C	8
Tavily results per call R_{tav}	5
Tavily search depth	basic
Per-result truncation length	8 000 characters
Tavily quick-answer	disabled
δ (Tavily cutoff offset)	1 day

The search tool exposes only a query-string parameter to the model; the cutoff date $\chi_q = \tau_q - \delta$ is hard-coded by `react._compute_end_date` and not modifiable through the tool interface. Two per-slug exceptions adjust the request body without affecting the conversation contract. Reasoning-class slugs whose lowercased name contains a substring from `LLM_REASONING_MODEL_PATTERNS` ({o1, o3, o4, r1, qwq}) skip the `temperature` and `top_p` fields, since the corresponding provider endpoints reject custom sampling parameters with HTTP 400. The per-slug override `MODEL_MAX_TOKENS_PARAM` sends `max_completion_tokens` instead of `max_tokens` for the OpenAI o-series and GPT-5 endpoints whose APIs require it; unlisted slugs keep `max_tokens`.

The ReAct loop layered on top of the rendered user prompt stacks four protocols, each fingerprinted independently from the prompt-template hash. The reflection scaffold tails a six-stage methodology onto the user prompt that asks the model to decompose the question, list at least three retrieval angles, reflect after each search, cross-validate with independent sources, run a contrarian self-check, and state confidence, with its SHA-256 logged in `run_meta.reflection_protocol_hash`. Budget awareness adds a footer announcing the global step cap T and search cap C before any inference begins, so the model never has to count messages to know its budget. Forced finalisation strips tools at the final step and emits a soft reminder one step earlier, with the lookahead window held at two steps under the default `REACT_FORCE_FINAL_ANSWER_LOOKAHEAD`. Tool revocation empties the tool list once cumulative `web_search` reaches C and `REACT_BUDGET_EXCEEDED_DROP_TOOLS` is true, after which the model can only emit content turns.

The soft minimum-search-count fallback `REACT_MIN_SEARCH_CALLS` is held at zero so the reflection scaffold drives investigation depth, and the probability/belief protocol is disabled, so the structured `<belief>...</belief>` block is not requested and the probability family in Section B.8 cannot be computed for this run.

Every harness-injected user message in the loop carries the same status header,

```
[Harness status] step k/N (R remaining) · web_search s/C used (M left).
```

followed by the scenario-specific directive on a new line. The four runtime injection paths are mutually exclusive within a turn, ordered by descending priority, and listed in Table 13.

Tool errors that arise inside the assistant-tool cycle cannot trigger a user message without breaking message ordering, so the same status text is emitted as a JSON `status` field inside the tool message payload, surfacing the live counters to the model without inserting an out-of-band turn. The bail-out retry switch `REACT_FINAL_ANSWER_RETRY` is disabled in this run, since the in-loop hard cutoff already covers the empty-final-row corner case it was designed for.

Table 13: Runtime injection paths inside the ReAct loop. Conditions are evaluated at the top of every iteration; at most one path fires per turn, and LOOKAHEAD denotes REACT_FORCE_FINAL_ANSWER_LOOKAHEAD (default 2).

Path	Trigger condition	Tool list this turn	Directive
Last-step hard cutoff	$\text{step} = N$	empty	emit <code>\boxed{...}</code> with content only
Penultimate soft warning	$N - \text{step} < \text{LOOKAHEAD}$ and budget unspent	current schema	one more search permitted, otherwise commit
Search-budget commit notice	$\text{cumulative searches} = C$ and $\text{REACT_BUDGET_EXCEEDED_DROP_TOOLS}=\text{true}$	empty	continue without search; commit on this or next turn
Unboxed-content continuation	last assistant turn returned content without parseable <code>\boxed{...}</code> and none above fired	current schema	resume reasoning or commit <code>\boxed{...}</code> now

The Tavily API-key parameter accepts multiple values, dispatched by `tavily_keys.TavilyKeyPool` under a least-used policy. Per-key state evolves under three failure kinds, summarised in Table 14.

Table 14: Tavily key-pool failure semantics. The cooldown duration T_{cool} defaults to 60 s; the network-retry budget is SEARCH_RETRY_MAX with backoff sequence SEARCH_BACKOFF_S from Table 20.

Failure kind	Trigger	Pool action
auth	HTTP 401 / 403	permanent blacklist; immediate swap to the next healthy key, no backoff, no quota consumption
rate_limit	HTTP 429 or quota-exceeded body	cooldown for T_{cool} on this key; immediate swap to the next healthy key without consuming the network-retry quota
other	5xx, network exception, non-JSON 200	no blacklist; consume one of SEARCH_RETRY_MAX attempts under SEARCH_BACKOFF_S

The pool instance is shared across grid cells in the same process via the module-level cache keyed on `tuple(TAVILY_API_KEY)`, so usage counts accumulate across cells rather than per-cell. Reflection, budget-awareness, and forced-finalisation texts are runtime values appended to the static template; the static prompt-template file, whose SHA-256 is logged in run metadata via `prompt_templates_hash`, is byte-identical to the protocol-free version, while the rendered user message, formed by the template extended with the runtime protocol additions, is recorded verbatim per sample in `run_results.user_prompt` for byte-level replay.

B.4 Detector configuration

The detector is independent of the model under test and may be replaced by any model capable of completing the task; if the detector base URL is empty, the detector falls back to LLM_BASE_URL. Table 15 lists the call parameters that distinguish the detector lane from the main LLM lane.

Table 15: Detector call parameters and how they differ from the main-LLM lane in Table 12.

Parameter	Detector value	Reason for divergence
Sampling temperature	0.0	deterministic, audit-style judgment
max_tokens	512	verdict and reason fit in ≤ 200 tokens
Timeout	60 s	detector calls are short; long timeouts inflate per-question latency
Concurrency cap	5	matches SEARCH_MAX_CONCURRENCY so per-question latency does not balloon
Retries / backoff	3 / [2, 5, 15] s	shorter than main-LLM retry chain because the verdict path is not on the critical answer path
Fail action	drop	fail-closed default: when the verdict cannot be obtained, remove the candidate item rather than admit it

Algorithm 4 Detector verdict parser `leak_filter._parse_verdict`. Returns None on any structural defect so the caller can apply the retry budget; never raises.

Input: Detector reply text t

Output: (verdict, reason) with verdict $\in \{\text{keep}, \text{drop}\}$, or None

```

1:  $t \leftarrow \text{strip}(t)$ 
2: if  $t$  is empty then
3:   return None
4: end if
5:  $C \leftarrow []$   $\triangleright$  ordered candidate JSON snippets
6: if  $t$  starts with { and ends with } then
7:   append  $t$  to  $C$   $\triangleright$  whole-body candidate
8: end if
9: scan  $t$  from the first { and append the substring closing at brace-depth 0 to  $C$   $\triangleright$  embedded-object candidate
10: for each candidate  $c$  in  $C$  do
11:   try  $o \leftarrow \text{json.loads}(c)$ ; on decode error continue
12:   if  $o$  is not a dict then
13:     continue
14:   end if
15:    $v \leftarrow o.\text{verdict}$ ;  $r \leftarrow o.\text{reason}$  or ""
16:   if  $v \notin \{\text{keep}, \text{drop}\}$  then
17:     return None
18:   end if
19:   return ( $v$ ,  $\text{str}(r)$ )
20: end for
21: return None

```

The detector input field whitelist covers six fields, with the question object, candidate options, and original question text intentionally excluded. Table 16 contrasts the whitelist against the items that never reach the detector.

Table 16: Detector input whitelist. The exclusion side prevents the detector from being framed as an answer auditor and avoids second-order leakage.

Whitelisted (per item)	Excluded by design
title, url, published_date, content, raw_content, cutoff_date	question stem x_i , candidate options $\mathcal{A}_i$, ground truth Y_i , all main-LLM dialogue turns, all run-level identifiers

The detector prompt enforces six explicit constraints embedded in the template. The `cutoff_date` is rendered as ISO 8601 in the prompt body. Specific, scheduled, and speculative future events are all classified as leakage, so a page describing “the launch is scheduled for 2026-08-15” against a 2026-04 cutoff drops alongside one describing the launch as already completed. The drop verdict is the default under doubt, and knowledge external to the provided text is forbidden as a basis for judgment, so the detector cannot rely on its own training corpus to second-guess the audited body. The output is a single JSON object on one line, with no surrounding prose. The words *question*, *answer*, and *options* never appear in the prompt body, which keeps the detector framed as a temporal auditor rather than an answer auditor.

The verdict parser walks the detector reply with the rule in Algorithm 4: pure JSON or a JSON object embedded in prose are both accepted, and any structural defect re-enters the retry loop instead of raising.

Verdict outcomes drive the retain/drop decision per result. Table 17 maps detector failure modes to harness behaviour.

When the detector returns drop, the entire result is removed because title, URL, content, and raw content all derive from the same page; partial retention would still leak through one of these channels. The SHA-256 of the detector prompt template is computed at startup by `leak_filter._compute_prompt_hash` and recorded in `run_meta.config_snapshot.leak_detector_prompt_hash` alongside `leak_detector_enabled` and `leak_detector_model`. The audit metadata is persisted on the corresponding entry of `run_results.search_calls` and never on `messages_trace`, so the main LLM cannot read verdict reasons from any past dialogue. Table 18 lists the audit fields.

When every item from one Tavily call is dropped, the synthesised `answer` field is also cleared, since the Tavily summary derives from the same set of pages and would otherwise leak by proxy. The algorithmic layer remains bounded by

Table 17: Detector failure handling under LEAK_DETECTOR_FAIL_ACTION=drop (default). Auth and parse failures are caught locally by `leak_filter._detect_one` so a misconfigured detector key never aborts the run; transient errors consume the retry budget under LEAK_DETECTOR_BACKOFF_S.

Failure	Retry?	Final action
HTTP 401 / 403	no	drop the item; record <code>detector_error_kind="auth"</code> in the audit
HTTP 5xx / network / timeout	yes, up to 3 tries with backoff [2, 5, 15] s	drop the item if retries exhausted
HTTP 200 with non-JSON body or invalid verdict	yes, same budget	drop the item if retries exhausted
HTTP 4xx other than 401 / 403	no	drop the item; record kind in audit

Table 18: Audit metadata persisted per Tavily call when the detector is active. Length invariants ensure forensic reconstruction: `detector_verdicts` and `published_dates_raw` both have length `n_results_raw`, while `n_results_kept` matches the post-filter result list seen by the main LLM.

Field	Content
<code>n_results_raw</code>	total results returned by Tavily before the detector
<code>n_results_kept</code>	post-detector count, equal to <code>n_results</code> on the LLM-visible payload
<code>detector_verdicts</code>	per-item list of keep, drop, or failed: <kind> in raw order
<code>detector_latency_ms</code>	wall-clock for the detector step on this Tavily call
<code>detector_error_kind</code>	first failure kind across items, or None when all succeeded
<code>published_dates_raw</code>	per-item <code>published_date</code> in raw order, length = <code>n_results_raw</code>

Tavily metadata accuracy: when a page’s `published_date` is missing or wrong, the date filter may miss some leakage, which the detector then catches at body level.

B.5 Leakage-mitigation scheme comparison

Table 19 contrasts the three search-content schemes used in this work against the no-barrier baseline and the dataset-level manual annotation. The residual leakage rate is the leak-conditional pass-through rate $FN/(TP + FN) = 1 - \text{recall}$, the fraction of real leaks that still reach the main LLM after the indicated filter.

Table 19: Three leakage-mitigation schemes versus the no-barrier and manual-annotation baselines. The residual leakage rate is the leak-conditional pass-through rate $FN/(TP + FN) = 1 - \text{recall}$.

Scheme	None	Tavily Date Restriction	Semantic LLM Annotation	Manual Annotation
Intervention layer	None	Algorithmic	Semantic	Dataset
Marginal cost	0	0	+1 LLM call per result	High human-labor cost
Residual leakage rate	100%	3%–16%	1%–1.5%	0%

Two quantitative entries in Table 19 need a derivation. The semantic-detector scheme adds one detector call per Tavily result, capped at $R_{\text{tav}} \cdot C = 5 \cdot 4 = 20$ calls per question under the run configuration of Table 12. With detector `max_tokens` held at 512 (Table 15) and the audit-set token-cost averaged across the three audit models, the marginal LLM-side spend stays below \$0.001 per question on the detector slug used in this run, two orders of magnitude below the manual-annotation floor. The residual band 1%–1.5% matches the leak-conditional pass-through measured in Section 4.3.4; the wider band 3%–16% for Tavily-only reflects items whose body explicitly references post-cutoff events under correct `published_date`. BLF [8] stacks a four-layer date-leakage defense around a single backtest; OracleProto and BLF defend complementary objects, the single inference and the single reproducible corpus, so the two schemes stack when an instantiated benchmark is paired with a calibrated forecaster.

The audit sample is constructed under a fixed protocol so the estimate in Table 6 carries a calibrated denominator. The denominator factors as

$$N = |\mathcal{M}_{\text{audit}}| \cdot Q_{\text{audit}} \cdot K_{\text{audit}} \cdot L_{\text{audit}} = 3 \cdot 30 \cdot 3 \cdot 1 = 270, \quad (35)$$

with the four factors fixed before sampling. The model set $\mathcal{M}_{\text{audit}} = \{\text{Kimi K2.5, Qwen3.5-Flash, GLM 5}\}$ takes three entries from the panel of Table 2 that span low, mid, and high search-call rates. The per-model question count $Q_{\text{audit}} = 30$ is sampled uniformly at random from $\mathcal{D}_M^{\text{pred}}$, and the trial budget $K_{\text{audit}} = 3$ matches the main-experiment trial budget. Each test item contributes $L_{\text{audit}} = 1$ search result per run, sampled uniformly at random from the up to $R_{\text{tav}} \cdot C = 20$ candidates of each test item.

Detector verdicts come from an independent Qwen3.5-Flash instance running the detector prompt fingerprinted in `config_snapshot.leak_detector_prompt_hash`, and human labels come from two annotators with cross-validation, so each verdict in the confusion matrix in Table 6 reflects two independent judgments before collapse.

B.6 Concurrency and error classification

Three async channels run concurrently inside one evaluation: the main LLM, the Tavily search wrapper, and the detector LLM. Table 20 states the per-channel concurrency cap and the retry budget consumed when a transport-layer failure surfaces from `httpx`; the backoff sequences are calibrated against per-channel failure modes (network jitter is short and bursty, rate limits are long and provider-paced).

Table 20: Per-channel concurrency and retry parameters. The backoff sequence is consumed in order, one entry per attempt, and reset only across distinct samples.

Channel	Concurrency	Retries	Backoff sequence (s)
Main LLM	5	5	network [2, 5, 15, 30, 60]; rate-limit [10, 30, 60, 120, 300]; 5xx [5, 15, 30, 60, 120]
Tavily search	5	3	[2, 5, 15]
Detector LLM	5	3	[2, 5, 15]

Once the backoff sequence is exhausted on a channel, the failure is reported to the harness and classified by `errors.classify` into one of seven kinds. Table 21 states the dispatch; the matching order is deliberate, with `content_policy` adjudicated before the generic `bad_request` so a moderation rejection is never silently grouped with a malformed request.

Table 21: Error classification by `errors.classify` and the resulting harness behaviour. AUTH cancels the run; `content_policy` matches before the generic `bad_request`.

Kind	Source	Harness response
network	<code>httpx.ConnectError / ReadTimeout / ConnectTimeout / WriteTimeout / WriteError / PoolTimeout / RemoteProtocolError; asyncio.TimeoutError</code>	retry under <code>LLM_BACKOFF_NETWORK_S</code>
rate_limit	HTTP 429 with optional <code>Retry-After</code> header	retry under <code>LLM_BACKOFF_RATE_LIMIT_S</code> ; honour <code>Retry-After</code> when present
server_5xx	HTTP 500–599	retry under <code>LLM_BACKOFF_SERVER_5XX_S</code>
auth	HTTP 401 / 403	raise <code>AuthError</code> , cancel in-flight tasks, abort the run
content_policy	HTTP 400 with body matching <code>{content_policy, content_filter, safety, data_inspection_failed, inappropriate_content, sensitive}</code>	no retry; record <code>error="content_policy"</code>
bad_request	HTTP 400 not matching the policy needles	no retry; record <code>error="bad_request"</code>
unknown	anything else	no retry; record <code>error="unknown"</code>

The downstream effect on aggregation follows three rules. Call errors and training-cutoff exclusions are dropped from the denominator, so they do not penalise the model. A parsing failure absent a call error contributes $c_{q,j} = 0$, since the format gate is strict and a non-boxed terminal turn is treated as a wrong answer rather than a missing one. Missing buckets in any composite are dropped and the remaining weights renormalised, so a metric undefined on a given bucket (such as the multi-only Hamming score on the yes/no bucket) does not silently inflate the global average with a degenerate zero.

B.7 Run layout and reproducibility envelope

Each evaluation lands in a self-contained run directory whose layout is fixed:

```
{RUNS_ROOT}/{run_id}/
  manifest.json
  db/
    <model_slug>.db          # one SQLite per virtual model slug
  analysis/                  # post-hoc CSV / MD / JSON
  logs/
    {run_id}.log
```

Three naming rules keep the directory uniquely addressable across runs and across grid cells. The `run_id` follows the format `YYYYMMDD-HHMMSS-xxxx` with `xxxx` a four-character hex suffix, so concurrent runs at the same wall-clock second do not collide. Model-slug filenames map `/` to `__` and any other character outside `[A-Za-z0-9._-]` to `_`, so two slugs collide only when their canonical forms already coincide. Grid cells are encoded under the virtual slug `{real}::r{R}::c{C}`, where `R` and `C` index the round cap and search cap of the cell; the runner peels the `::r::c` suffix before dispatching the LLM call, while `db.parse_virtual_slug` reverses the encoding for the analysis pass.

Hash chain. Every byte-meaningful artefact in a run carries a fingerprint inside `run_meta` or at the top of `manifest.json`. Table 22 lists the seven covered artefacts; any byte-level change to the corpus, the templates, the detector prompt, or the harness configuration shifts at least one hash, and the change becomes visible to every downstream consumer that opens `run_meta` or `manifest.json`.

Table 22: Reproducibility hash chain. `config_snapshot` additionally redacts API keys via `db.snapshot_settings`: every key whose name contains `API_KEY` is replaced by its first four characters concatenated with the sha256-12 of the full key, so a leaked snapshot exposes neither the key nor the count of keys per provider.

Hash	Covers
<code>source_db_hash</code>	sha256 of the source SQLite file at <code>SOURCE_DB</code>
<code>metadata_hash</code>	sha256 of the canonicalised <code>features.json</code> payload
<code>prompt_templates_hash</code>	sha256 of the canonical key=value form of the rendered template body
<code>reflection_protocol_hash</code>	sha256-16 of the appended reflection scaffold; None when the protocol is disabled
<code>belief_protocol_hash</code>	sha256-16 of the appended belief block; None when the protocol is disabled
<code>leak_detector_prompt_hash</code>	sha256-16 of the detector prompt template, written into <code>config_snapshot</code>
<code>config_snapshot</code>	redacted Settings dict; API-key fields reduced to prefix and sha256-12

Resume protocol. Reusing a `run_id` re-enters the same directory and replays the writer state. A sample (q, j) counts as accounted-for iff

$$s\{j\}_{\text{created_at}} \neq \text{NULL} \wedge s\{j\}_{\text{error}} \in \{\text{NULL}, \text{"skipped_training_cutoff"}\}, \quad (36)$$

so error-free completed rows and training-cutoff-excluded rows are not retried, while any other error code is re-emitted on the next run that reuses the same `run_id`. Cutoff-skipped rows are pre-seeded into the writer queue ahead of LLM work, so the planner counters `[done/total]` stay predictable from the first log line.

Per-sample columns. Beyond the conversation trace and the standard `q_id`, `model`, `cutoff`, `delta`, `created_at`, and `error` fields, each row carries three groups of provider- and protocol-specific markers. Six response-marker columns (`finish_reason`, `nudges_used`, `step_metrics`, `response_id`, `system_fingerprint`, `service_tier`) copy provider-side metadata verbatim from the chat-completions payload, used for forensic checks against silent provider drift. Three belief-protocol columns (`belief_final`, `belief_trace`, `belief_parse_ok`) are populated only when the belief block is requested and otherwise left `NULL`, so toggling the protocol does not change the schema. A single forced-finalisation column `final_answer_retry_used` is set when the in-loop hard cutoff fires its bail-out retry, letting the post-hoc analysis split rows by whether the boxed answer arrived through the normal path or through the retry escape.

`init_schema` introspects `PRAGMA table_info` on connection and emits one `ALTER TABLE ADD COLUMN` per missing column, so a brand-new database and a database missing any subset of these columns both reach the full column set

in a single startup pass without manual migration. The complete messages trace per sample is persisted by default and can be disabled via `WRITE_MESSAGES_TRACE=false`, dropping database size by approximately 80% at the cost of removing the byte-level dialogue replay; the standalone .db then still carries every aggregate-recomputable column.

The software stack pins Python ≥ 3.12 , pydantic 2 / pydantic-settings, and embedded SQLite; exact versions are in the repository lockfile.

B.8 Other metrics implemented but not utilized

Three metric families exist as columns in the analytics summary file but do not surface in the main results, since the run configuration disables the probability/belief protocol on which two of the three families depend. Table 23 groups them by gating condition.

Table 23: Three implemented metric families and the protocol that gates them. Family A is computed in this run; families B and C aggregate to None.

Family	Gate	Visible columns when gate is open
A. Discrete-native	boxed letter set parses, $\widehat{G}_{i,M} \neq \perp$	Hamming, MV-Acc, $\bar{H}$, VCI, entropy-accuracy buckets
B. Probabilistic	belief vector $\hat{p}_{q,j}$ emitted alongside the box	BS^{lab} , BS^{dec} , NLL, MBS, BI, ABI
C. Behavioural-analysis	per-step belief trace persisted	belief-evolution group, reflection A/B, tool-use PD, calibration cross-tab

A. Discrete-native family. The five quantities in this family derive from the boxed letter set alone. The Hamming partial credit measures per-trial overlap between the predicted and gold letter sets,

$$\text{Hamming}_{q,j} = 1 - \frac{1}{k_q} \sum_{\ell \in \mathcal{O}_q} |\mathcal{K}[\ell \in \hat{S}_{q,j}] - \mathcal{K}[\ell \in G_q]|, \quad (37)$$

aggregated over MC-multi only since single-choice partial credit collapses to strict 0/1 already covered by $\text{pass}@1_{\text{avg}}$. The majority-vote accuracy is the fraction of questions whose modal letter set matches G_q , reported alongside the gain $\text{MV-Acc} - \text{pass}@1_{\text{avg}}$, with a winner-uniqueness rule dropping any question whose top vote is tied so a model never wins by tie-breaking. The mean predictive entropy

$$\bar{H} = \frac{1}{|\mathcal{D}|} \sum_q H_q, \quad (38)$$

takes H_q as the Shannon entropy of the intra-question vote distribution for single-answer questions and the mean per-label binary entropy across $\mathcal{O}_q$ for multi-answer questions. The vote-concentration index

$$\text{VCI}_q = \max_{\ell} \frac{n_{q,\ell}}{K_q^{\text{eff}}}, \quad (39)$$

gives the fraction of trials voting for the modal letter set, reported per question and as a mean over $\mathcal{D}$. The entropy-accuracy three-bucket diagnostic splits each model's questions into three entropy buckets at per-model cut-points and reports mean accuracy within each, with cross-model bucket cells intentionally not comparable since the cut-points differ.

B. Probabilistic family. Each per-question score requires the model to emit a belief vector $\hat{p}_{q,j} \in [0, 1]^{k_q}$ alongside `\boxed{...}`. Single-answer questions constrain the simplex via $\sum_{\ell} \hat{p}_{q,j,\ell} = 1$ within tolerance 10^{-3} , and multi-answer questions emit independent Bernoulli probabilities. The parser `parse_belief` validates both regimes before any score is computed, and probabilities are clipped to $[\epsilon, 1 - \epsilon]$ with $\epsilon = 10^{-3}$ before any logarithm. The per-label Brier averages the squared error across the k_q candidate labels with $o_{q,j,\ell} \in \{0, 1\}$ the per-label outcome,

$$\text{BS}_{q,j}^{\text{lab}} = \frac{1}{k_q} \sum_{\ell} (\hat{p}_{q,j,\ell} - o_{q,j,\ell})^2, \quad (40)$$

and is reported on both single- and multi-answer questions. The decision-mass Brier rescales by k_q to match the textbook two-class Brier on the yes/no bucket and is defined for single-answer questions only,

$$\text{BS}_{q,j}^{\text{dec}} = k_q \cdot \text{BS}_{q,j}^{\text{lab}}. \quad (41)$$

The negative log-likelihood takes different forms for the two regimes,

$$\text{NLL}_{q,j} = \begin{cases} -\log p_{q,j,\ell^*}, & \ell^* = \arg \max_{\ell} o_{q,j,\ell} & \text{single,} \\ -\frac{1}{k_q} \sum_{\ell} [o_{q,j,\ell} \log p_{q,j,\ell} + (1 - o_{q,j,\ell}) \log(1 - p_{q,j,\ell})] & \text{multi.} \end{cases} \quad (42)$$

The mean Brier score gain rescales the log-score onto a 100-point axis that maps a chance prediction to zero, defined only for single-answer questions,

$$\text{MBS}_{q,j} = 100(\log_2 p_{q,j,\ell^*} + 1). \quad (43)$$

The model-level aggregates are the Brier index in Eq. (32), the mean NLL, the mean MBS, and two adjusted Brier indices **abi-crowd** and **abi-uniform** that subtract a per-question baseline γ_q before applying the sign-symmetric square root in Eq. (33). **abi-uniform** uses

$$\gamma_q^{\text{uni}} = \frac{1}{k_q} \sum_{\ell} (1/k_q - o_{q,\ell})^2, \quad (44)$$

the closed-form per-label Brier of the uniform predictor; **abi-crowd** replaces the uniform vector by the leave-one-out mean across the other tested models, falling back to **abi-uniform** when only one model has a probability vector on that question. The probability protocol is disabled in this run, so the entire family aggregates to None on every model and the corresponding columns appear as blank cells in `per_model_summary.csv`.

C. Behavioural-analysis family. Four diagnostic groups depend on the per-step belief trace stored in `s{j}_belief_trace`. The belief-evolution group reports four trial-level scalars that characterise how the belief vector b_t moves across reasoning steps,

$$V_{q,k} = \frac{1}{T-1} \sum_{t=2}^T \|b_t - b_{t-1}\|_2 \quad (\text{step-to-step volatility}), \quad (45)$$

$$\sigma_q = \text{std}_k(b_T^{(q,k)}) \text{ around the centroid of valid finals} \quad (\text{inter-trial variance}), \quad (46)$$

$$C_{q,k} = \min\{t : \|b_T - b_t\|_2 < 0.05\} \quad (\text{convergence step}), \quad (47)$$

$$\eta_{q,k} = \frac{\text{NLL}(b_0) - \text{NLL}(b_T)}{\max(1, \text{search_calls})} \quad (\text{evidence efficiency}). \quad (48)$$

A binary counter-evidence engagement flag is emitted alongside, set when the last `counterevidence` bullet in the trace mentions an option letter outside the boxed answer. The reflection A/B group pairs runs that match on every fingerprint in Table 22 except `reflection_protocol_hash`, then reports paired-bootstrap 95% CIs of ΔBI , $\Delta\sigma$, ΔC , $\Delta\eta$ with sign $\Delta = \text{on} - \text{off}$, optionally stratified by question type. The tool-usage partial dependence fits a hand-rolled IRLS logistic for $\Pr(\text{correct} \mid x)$ and a closed-form ridge regression for $\mathbb{E}[\text{NLL} \mid x]$ on the five-feature input vector x comprising `tool_calls_count`, `react_steps`, `latency_ms`, `prompt_tokens`, and `completion_tokens`, z-scored before fitting and refit per model. The confidence-calibration diagnostic cross-tabulates a self-reported linguistic three-bin confidence (low / medium / high) emitted by the model in the trace against a numeric ten-bin discretisation of $\max_{\ell} p_{q,j,\ell}$ computed from the belief vector at decision time, with a conflict flag firing for any model whose two signals disagree under either of the thresholds $\bar{p}_{\text{low}} > 0.70$ on the *low* bucket (linguistic low but numeric high) or $\bar{p}_{\text{high}} < 0.55$ on the *high* bucket (linguistic high but numeric low).

All four groups inherit the gate of family C in that the belief trace is emitted only when the belief block is requested. Turning the protocol on for a future run unlocks every column above without further code change, since the analysis layer dispatches on the manifest field `analysis_schema` and the persisted column is a JSON list rather than a fixed-width vector.